Tutoriat LMC

Vlăduț Ghiță, Raluca Ionete, Voaideș Robert

April 22, 2026

Contents

1 Introducere în LP și Prolog **3**
- 1.1 Administrativ . . . 3
- 1.2 De ce Logică? . . . 3
- 1.3 Logica Propozițională - intro . . . 4
  - 1.3.1 Principiul inducției pe formule . . . 5
  - 1.3.2 Principiul recursiei pe formule . . . 6
- 1.4 Semantica LP . . . 7
  - 1.4.1 Evaluări . . . 7
  - 1.4.2 Modele. Tautologii . . . 8
- 1.5 Prolog Basics . . . 11

2 Continuare Prolog, FNC si FND **14**
- 2.1 Funcții Boolene . . . 14
- 2.2 Forma normală Conjunctivă și Forma Normală Disjunctivă . . . 15
  - 2.2.1 Concepte Fundamentale . . . 15
  - 2.2.2 FNC . . . 15
  - 2.2.3 FND . . . 15
  - 2.2.4 Aducerea formulelor în FNC și FND . . . 16
  - 2.2.5 Algoritmul de construcție pentru FNC și FND de la un tabel . . . 16
  - 2.2.6 Algoritmul de aducere a unei formule la FND/FNC . . . 17
- 2.3 Prolog Aritmetica si Exercitii . . . 19

3 Clauze, Rezoluție și Prolog **23**
- 3.1 Continuare FNC și FND . . . 23
- 3.2 Clauze și forma clauzală . . . 25
  - 3.2.1 Definiții de bază . . . 25
  - 3.2.2 Legătura dintre clauze și FNC . . . 26
  - 3.2.3 Conversia inversă: din clauze la formula φ_S . . . 27
- 3.3 Rezoluția pe clauze . . . 28
- 3.4 Algoritmul Davis–Putnam (DP) . . . 29
- 3.5 Prolog findall/setof/bagof . . . 32

4 Sintaxa LP. Rezoluție și Prolog **37**
- 4.1 Termeni compusi in Prolog . . . 37
- 4.2 Sintaxa LP . . . 39
  - 4.2.1 Sistemul de deducție . . . 39
  - 4.2.2 Γ-teoreme . . . 39

4.2.3 Γ-demonstrații 40
4.2.4 Proprietăți sintactice 42

Chapter 1

Introducere în LP și Prolog

1.1 Administrativ

- Cursurile d-lui Leuștean sunt foarte bine scrise și chiar merită citite pentru a învăța la materia asta. Le găsiți pe toate pe GitHub. În plus, la examen **o să fie nevoie** să folosiți propoziții din curs (care sunt numerotate), și să le citați, așa că vă ajută să știți cam care este structura cursului. Nu o să reluăm teoria pe care o prezintă în curs **decât** dacă ne cereți asta.
- Seminariile sunt destul de complexe, acoperă ok materia, iar soluțiile sunt postate pe site, așa că noi o să încercăm să lucrăm la tutoriat exerciții extra.
- Puteți să ne scrieți oricând în privat/pe grup cu întrebări/sugestii/idei, iar noi o să încercăm să vă ajutăm și să le punem în practică. De asemenea, puteți să ne sugerați ce ați refera să lucrăm la tutoriat.
- O să publicăm pe GitHub tutoriatele săptămânal și o să anunțăm, prin README, ce o să facem la următorul tutoriat. De asemenea, plănuim să creăm liste de exerciții, la care să publicăm și rezolvările. Dacă o să doriți să ne uităm pe rezolvările voaste, puteți să ne ziceți.

1.2 De ce Logică?

Este foarte normal să vă întrebați de ce e nevoie să studiați logica ca studenți la info, având în vedere că e aproape matematică pură, care nu pare să aibă foarte mare aplicabilitate în viața reală.
Well, sunt mai multe motive.

Logica e baza informaticii

Aproape toți oamenii care au fondat informatica erau logicieni. Tot ce înseamnă informatică formală se poate traduce în logică, literally.

Asta se traduce în două lucruri:

- nu o să scăpați de logică pentru încă ceva timp, în anul II aveți două materii care se construiesc pe logică și la care o să vă ajute foarte mult să aveți bazele bine stabilite.
- felul foarte formal de a gândi care este promovat de logică vă va ajuta foarte mult în a deveni programatori mai buni; o să ajungeți să gândiți mult mai mult ce scrieți, atât în matematică cât și în programare, pentru a fi mult mai riguros și mai logic exprimat, ceea ce o să vă ajute mai ales atunci când aveți lucruri mai complicate de făcut.

Logica vă deschide drumul și către o ramură destul de nișată a informaticii, dar care este foarte bine plătită și foarte AI-proof, anume verificarea formală, care, pe scurt, presupune demonstrarea matematică a corectitudinii unui cod, lucru care este esențial în sisteme high-risk (procesoare, sisteme medicale, rachete etc.) Facultatea noastră este foarte bogată în logicieni, se țin foarte des seminarii de logică la ILDS sau LOS, așa că dacă vreți să faceți cercetare în timpul facultății sau să vă specializați pe partea de verificare formală sau alte tipuri de logică, chiar aveți cu cine.

1.3 Logica Propozițională - intro

Formulele stau la baza Logicii Propoziționale și sunt principalul lucru cu care o să lucrați capitolul ăsta. Cursul menționează că ele sunt definite inductiv:

Definition 1 (Formule). Formulele sunt definite astfel:

(1) Orice variabilă propozițională este formulă. (Adică $\forall v \in V$, v este formulă)

(2) Dacă φ este formulă, atunci $\neg\varphi$ este formulă

(3) Dacă φ, ψ sunt formule, atunci $\psi \to \varphi$ este formulă

Dar de ce este inductiv? Până acum, ați folosit doar inducția pe numere naturale, care știți cum funcționa. Aveam un caz de bază $P(0)$ și, presupunând că predicatul era adevărat pentru un n natural, construiam următorul caz, $n+1$, și demonstram corectitudinea.

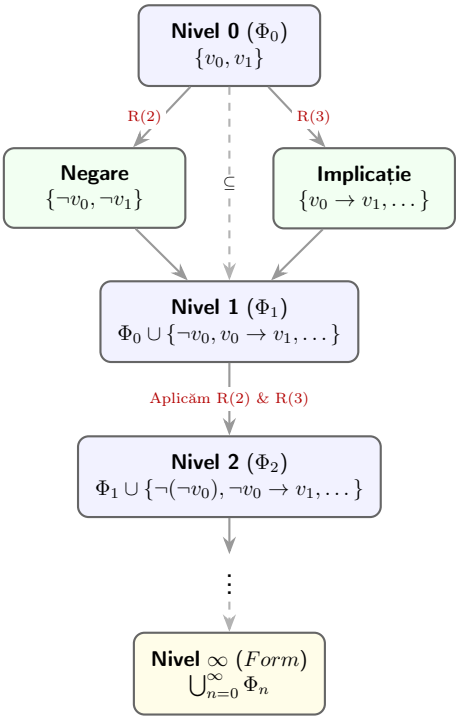
Practic, inducția reprezintă metoda de a pleca de la ceva particular și de a generaliza incremental. Pentru numere naturale, pasul incremental este simplu, anume adunarea.

Noi putem extinde metoda asta, astfel, și la formule. Avem cazul particular, pe care îl definim a fi formulă, adică variabilele v. De aici, incremental, construim toate celelalte formule. În loc de un singur pas incremental, avem doi:

$\neg$ putem nega formula noastră curentă pentru a obține o nouă formulă

$\to$ atunci când avem două formule, o nouă formulă este definită prin implicație

Această metodă se numește **inducție structurală**.
În următoarea figură puteți vedea cum sunt construite formulele din doar două variabile.



1.3.1 Principiul inducției pe formule

Același principiu poate fi a aplicat pentru a demonstra că orice formulă respectă o proprietate $\mathbf{P}$ (e.g., orice formulă are mai $\leq$ conectori logici decât variabile, orice formulă este bine parantezată, etc.).

Proposition 1. Fie $\mathbf{P}$ o proprietate. Presupunem că:

(0) Orice variabilă v are proprietatea $\mathbf{P}$.

(1) Pentru orice formulă φ, dacă φ are proprietatea $\mathbf{P}$, atunci $\neg\varphi$ are proprietatea $\mathbf{P}$.

(2) Pentru orice formule φ, ψ, dacă acestea au proprietatea $\mathbf{P}$, atunci $\varphi \to \psi$ are proprietatea $\mathbf{P}$.

Atunci orice formulă φ are proprietatea $\mathbf{P}$.

Example. Vrem să demonstrăm că orice formulă φ este parantezată corect. Formal: orice formulă φ are proprietatea $\mathbf{P}(\varphi) : st(\varphi) = dr(\varphi)$, unde $st(\varphi)$ este numărul de paranteze "(", iar $dr(\varphi)$ este numărul de paranteze ")".

Proof. Demonstrăm proprietatea $P(\varphi)$ prin inducție structurală după construcția formulei φ.

Cazul de bază: $\varphi = v$, unde $v \in V$ este o variabilă propozițională. Conform definiției sintaxei, variabilele nu conțin paranteze.

$$st(v) = 0 \text{ și } dr(v) = 0 \Rightarrow st(v) = dr(v)$$

Deci $P(v)$ este adevărată.

Pasul inductiv: Presupunem că proprietatea este adevărată pentru formulele ψ și χ (Ipoteza Inductivă). Deci $st(\psi) = dr(\psi)$ și $st(\chi) = dr(\chi)$.

Demonstrăm pentru regulile de construcție:

- **Cazul** $\varphi = (\neg\psi)$: Calculăm numărul de paranteze folosind definiția recursivă:

$$st(\varphi) = st(\psi) + 1$$
$$dr(\varphi) = dr(\psi) + 1$$

  Din Ipoteza Inductivă știm că $st(\psi) = dr(\psi)$. Adunând 1 în ambele părți, obținem $st(\varphi) = dr(\varphi)$.

- **Cazul** $\varphi = (\psi \to \chi)$: Calculăm numărul de paranteze:

$$st(\varphi) = 1 + st(\psi) + st(\chi)$$
$$dr(\varphi) = dr(\psi) + dr(\chi) + 1$$

  Înlocuind din Ipoteza Inductivă ($st \to dr$), avem:

$$st(\varphi) = 1 + dr(\psi) + dr(\chi)$$

  Datorită comutativității adunării, $1 + dr(\psi) + dr(\chi) = dr(\psi) + dr(\chi) + 1$, deci $st(\varphi) = dr(\varphi)$.

Conform principiului inducției structurale, proprietatea $P(\varphi)$ este adevărată pentru orice formulă φ. □

1.3.2 Principiul recursiei pe formule

Dacă inducția ne ajută să demonstrăm proprietăți, principiul recursiei pe formule este folosit pentru a da **definiții recursive** riguroase diverselor funcții asociate formulelor.

Proposition 2 (Principiul recursiei pe formule)**.** Fie A o mulțime și funcțiile:

$$G_0 : V \to A, \quad G_\neg : A \to A, \quad G_\to : A \times A \to A$$

Atunci există o **unică** funcție $F : Form \to A$ care satisface următoarele

proprietăți:

(R0) $F(v) = G_0(v)$ pentru orice variabilă $v \in V$.

(R1) $F(\neg\varphi) = G_\neg(F(\varphi))$ pentru orice formulă φ.

(R2) $F(\varphi \to \psi) = G_\to(F(\varphi), F(\psi))$ pentru orice formule φ, ψ.

Example. Vrem să definim o funcție $var : Form \to \mathcal{P}(V)$ care extrage mulțimea tuturor variabilelor propoziționale ce apar într-o formulă φ (unde $\mathcal{P}(V)$ este mulțimea părților lui V). O definiție recursivă a lui var este următoarea:

$$\begin{aligned} var(v) &= \{v\} && \text{pentru orice variabilă } v \\ var(\neg\varphi) &= var(\varphi) && \text{pentru orice formulă } \varphi \\ var(\varphi \to \psi) &= var(\varphi) \cup var(\psi) && \text{pentru orice formule } \varphi, \psi \end{aligned}$$

În acest caz, maparea pe principiul general se face astfel:

- Mulțimea A este $\mathcal{P}(V)$.
- $G_0 : V \to \mathcal{P}(V)$, definită prin $G_0(v) = \{v\}$.
- $G_\neg : \mathcal{P}(V) \to \mathcal{P}(V)$, definită prin $G_\neg(X) = X$ (negația nu modifică variabilele existente).
- $G_\to : \mathcal{P}(V) \times \mathcal{P}(V) \to \mathcal{P}(V)$, definită prin $G_\to(X, Y) = X \cup Y$ (reunim variabilele din ambele subformule).

1.4 Semantica LP

În acest capitol se studiază semantica, adică, informal, **ce înseamnă o formulă/cum îi dăm sens**. Acest lucru se face prin intermediul **evaluărilor** (sau interpretărilor, dar mai rar), care sunt funcții ce **dau valoare variabilelor propoziționale**.

1.4.1 Evaluări

O evaluarea este orice funcție:

$$e : V \to \{0, 1\}$$

Cu ajutorul recursiei pe formule este foarte ușor să extindem de la evaluări (care sunt pe variabile), la o funcție care interpretează formule:

Theorem 1. Pentru orice evaluare $e : V \to \{0, 1\}$ există o unică funcție (vezi recursia pe formule):

$$e^+ : Form \to \{0,1\}$$

care verifică următoarele proprietăți

(1) $e^+(v) = e(v), \quad \forall v \in V$

(2) $e^+(\neg\varphi) = \neg e^+(\varphi), \quad \forall \varphi \in Form$

(3) $e^+(\varphi \to \psi) = e^+(\varphi) \to e^+(\psi), \quad \forall \varphi, \psi \in Form$

Două cum puteți vedea, conectorii logici $(\neg, \to)$ sunt îngroșați atunci când sunt scoși din e^+. Acest lucru este pentru a arăta că aceștia nu mai sunt simboluri, în cadrul formulelor, ci **operații** pe $\{0,1\}$, cu tabele de adevăr știute.

Proposition 3. Dacă două evaluări sunt identice pentru toate variabilele dintr-o formulă φ, atunci ele vor da aceeași evaluare a formulei.

$$e_1(v) = e_2(v)\ \forall v \in Var(\varphi) \Rightarrow e_1^+(\varphi) = e_2^+(\varphi)$$

Propoziția 3 este utilă prin faptul că, atunci când evaluăm formule, putem să ne restrângem doar la variabilele din formulele pe care le evaluăm. Acest lucru se traduce în **metoda tabelului**, în care fiecare linie a tabelului este o evaluare diferită a variabilelor din $Var(\varphi)$.

1.4.2 Modele. Tautologii

Definition 2. Fie o evaluare $e : V \to \{0,1\}$.

Model: e este **model** pentru o formulă φ dacă $e^+(\varphi) = 1$. Notăm $e \models \varphi$.

Satisfiabilitate: O formulă φ se numește **satisfiabilă** dacă există cel puțin o evaluare e care este model pentru φ (adică $\exists e$ a.î. $e \models \varphi$).

Nesatisfiabilitate: O formulă φ este **nesatisfiabilă** (sau **contradictorie**) dacă nu admite niciun model (adică pentru orice evaluare e, $e^+(\varphi) = 0$). Notăm uneori prin $\varphi \models \bot$.

Tautologie: O formulă φ este **tautologie** dacă orice evaluare este model pentru ea (adică $\forall e, e \models \varphi$). Notăm acest lucru prin $\models \varphi$.

Notation. Mulțimea tuturor modelelor unei formule φ se notează cu $Mod(\varphi)$.

$$Mod(\varphi) = \{e : V \to \{0,1\} \mid e \models \varphi\}$$

Toți termenii definiți în 2 sunt foarte importanți și trebuie reținuți, pentru că o să tot lucrați cu ei.

Example. Formula $p \vee \neg p$ este o **tautologie**, deoarece indiferent de valoarea lui p, formula va fi adevărată. În schimb, $p \wedge \neg p$ este o **contradicție**, deoarece nicio evaluare nu o poate face adevărată.

Proposition 4 (Legile lui De Morgan). Pentru orice formule $\varphi, \psi \in Form$, au loc următoarele echivalențe logice:

$$\neg(\varphi \vee \psi) \sim \neg\varphi \wedge \neg\psi \tag{10}$$
$$\neg(\varphi \wedge \psi) \sim \neg\varphi \vee \neg\psi \tag{11}$$

Demonstrație:

Vom demonstra prima lege: $\neg(\varphi \vee \psi) \sim \neg\varphi \wedge \neg\psi$. Demonstrația celei de-a doua legi se realizează în mod absolut analog.

(i) Metoda tabelului de adevăr

Fie $e : V \to \{0, 1\}$ o evaluare oarecare. Construim tabelul de adevăr pentru a evalua extensia e^+ pentru ambele formule. Coloanele de la stânga liniei verticale reprezintă valorile posibile pentru $e^+(\varphi)$ și $e^+(\psi)$, iar în dreapta evaluăm pas cu pas subformulele.

φ	ψ	$\varphi \vee \psi$	$\neg(\varphi \vee \psi)$	$\neg\varphi$	$\neg\psi$	$\neg\varphi \wedge \neg\psi$
1	1	1	0	0	0	0
1	0	1	0	0	1	0
0	1	1	0	1	0	0
0	0	0	1	1	1	1

Analizând tabelul, observăm că pentru toate cele 4 cazuri posibile ale unei evaluări e, valoarea sub coloana $\neg(\varphi \vee \psi)$ este identică cu valoarea sub coloana $\neg\varphi \wedge \neg\psi$. Astfel, $e^+(\neg(\varphi \vee \psi)) = e^+(\neg\varphi \wedge \neg\psi)$ pentru orice evaluare e, ceea ce înseamnă că au exact aceleași modele. Prin urmare, $\neg(\varphi \vee \psi) \sim \neg\varphi \wedge \neg\psi$.

(ii) Metoda prin raționament semantic (formal)

Raționăm direct. Pentru o evaluare arbitrară $e : V \to \{0, 1\}$, observăm că:

$$e^+(\neg(\varphi \vee \psi)) = \neg(e^+(\varphi) \vee e^+(\psi)),$$
$$e^+(\neg\varphi \wedge \neg\psi) = \neg e^+(\varphi) \wedge \neg e^+(\psi).$$

Avem cazurile:

(a) $e^+(\varphi) = 0$. Atunci

$$\neg(e^+(\varphi) \vee e^+(\psi)) = \neg(0 \vee e^+(\psi)) = \neg e^+(\psi),$$
$$\neg e^+(\varphi) \wedge \neg e^+(\psi) = \neg 0 \wedge \neg e^+(\psi) = 1 \wedge \neg e^+(\psi) = \neg e^+(\psi).$$

(b) $e^+(\varphi) = 1$. Atunci

$$\neg(e^+(\varphi) \vee e^+(\psi)) = \neg(1 \vee e^+(\psi)) = \neg 1 = 0,$$
$$\neg e^+(\varphi) \wedge \neg e^+(\psi) = \neg 1 \wedge \neg e^+(\psi) = 0 \wedge \neg e^+(\psi) = 0.$$

Deoarece în ambele cazuri rezultatele evaluărilor coincid, obținem că $e^+(\neg(\varphi \vee \psi)) = e^+(\neg\varphi \wedge \neg\psi)$ pentru orice evaluare e. Rezultă $Mod(\neg(\varphi \vee \psi)) = Mod(\neg\varphi \wedge \neg\psi)$, deci $\neg(\varphi \vee \psi) \sim \neg\varphi \wedge \neg\psi$. □

Exercițiu. Să se găsească câte un model (o evaluare de adevăr) pentru fiecare dintre următoarele formule:

(i) $\varphi_1 = (v_1 \vee v_2) \to \neg v_3$

(ii) $\varphi_2 = (v_0 \wedge v_1) \wedge \neg(v_1 \to v_2)$

(iii) $\varphi_3 = v_5 \leftrightarrow (v_6 \wedge \neg v_5)$

Soluție. Reamintim că un model pentru o formulă φ este o funcție de evaluare $e : V \to \{0, 1\}$ astfel încât $e^*(\varphi) = 1$.

(i) Pentru $\varphi_1 = (v_1 \vee v_2) \to \neg v_3$: O implicație este adevărată dacă antecedentul este fals sau consecventul este adevărat. Alegem un model e în care $e(v_1) = 0$ și $e(v_2) = 0$. Atunci:

$$e^+((v_1 \vee v_2) \to \neg v_3) = (0 \vee 0) \to e^+(\neg v_3) = 0 \to e^+(\neg v_3) = 1$$

Deci, un model posibil este orice funcție e cu $e(v_1) = 0, e(v_2) = 0$ (valorile pentru v_3 și restul variabilelor pot fi arbitrare).

(ii) Pentru $\varphi_2 = (v_0 \wedge v_1) \wedge \neg(v_1 \to v_2)$: Pentru ca o conjuncție să fie adevărată, ambii termeni trebuie să fie adevărați.

1. $e^+(v_0 \wedge v_1) = 1 \Rightarrow e(v_0) = 1$ și $e(v_1) = 1$.
2. $e^+(\neg(v_1 \to v_2)) = 1 \Rightarrow e^+(v_1 \to v_2) = 0 \Rightarrow e(v_1) = 1$ și $e(v_2) = 0$.

Modelul este $e(v_0) = 1, e(v_1) = 1, e(v_2) = 0$.

(iii) Pentru $\varphi_3 = v_5 \leftrightarrow (v_6 \wedge \neg v_5)$: Echivalența este adevărată dacă ambele părți au aceeași valoare de adevăr. Dacă presupunem $e(v_5) = 1$, atunci trebuie ca $e^+(v_6 \wedge \neg v_5) = 1$, ceea ce implică $e^+(\neg v_5) = 1 \Rightarrow e(v_5) = 0$ (contradicție). Dacă presupunem $e(v_5) = 0$, atunci trebuie ca $e(v_6 \wedge \neg v_5) = 0$. Cum $e^+(\neg v_5) = 1$, conjuncția $e^+(v_6 \wedge 1)$ este 0 dacă și numai dacă $e(v_6) = 0$. Deci, un model este $e(v_5) = 0, e(v_6) = 0$.

□

1.5 Prolog Basics

Prologul are la baza **rezolutia**, despre care o sa invatati mai tarziu in curs, dar pe scurt, se bazeaza pe a ajunge la un adevar. Noi cand scriem cod in prolog de fapt scriem la fiecare termen **un predicat** (o functie care returneaza bool), si la chestii mai complicate prologul practic cauta o cale catre un adevar prin predicatele pe care le definim noi.

> **Note** (Litere mari vs mici). Se face distinctie intre parametrii cu litere mari si litere mici:
>
> Cele cu litere mici se refera la lucruri concrete (numere, stringuri, etc.).
>
> Cele cu litere mari sunt variabile si ele de fapt pot reprezenta orice termen in spate, fie numere, stringuri, liste etc.
>
> In linia de comanda cand rulam programul daca dam variabile ca parametrii prolog ii va interpreta ca necunoscute si incearca sa gaseasca valori care respecta regulile pentru acele variabile.

Mai jos aveti un exemplu de niste predicate simple (fapte) si un predicat definit prin reguli:

Listing 1.1: Family relations in Prolog

```
% Cazurile de baza (oarecum ca la o functie recursiva)
parent(albert, bob).
parent(albert, betsy).
parent(albert, bill).

parent(alice, bob).
parent(alice, betsy).
parent(alice, bill).

% Reguli extra, prolog va incerca sa ajunga la un adevar pe care l-am definit mai sus
% Separarea prin virgula inseamna 'si' intre toate predicatele enumerate
sibling(X, Y) :-
    parent(Z, X),
    parent(Z, Y),
    X \= Y.
```

Este important sa distingem intre operatorii =, ==, 'is':

- Operatorul = de fapt este operatorul de unificare, prolog va incerca sa unifice cei doi termeni cand face backtracking sa caute solutii.
- Operatorul == comparare stricta, nu incearca sa unifice termenii.
- Operatorul 'is' evalueaza expresii matematice.

Exemple:

Listing 1.2: Distinctie operatorii in prolog

```
?- f(X, 2) = f(1, Y).
X = 1,
Y = 2.

?- f(X, 2) == f(1, Y).
false. % Aici se face comparare fara sa incerce sa unifice
    termenii X si Y

?- X = 2 + 3.
X = 2+3.

?- X is 2 + 3.
X = 5.

?- 5 == 2 + 3
false. % Aici termenul '5' nu este egal cu termenul '2 + 3'
```

Liste in Prolog

Listele arata vizual ca si in python: `[1, 2, abc, 4, 5]` dar modul in care lucram cu ele este diferit. In prolog ne folosim de o sintaxa cu Head si Tail:

Listing 1.3: Liste in prolog

```
?- [H|T] = [1, 2, 3, 4, 5].
H = 1,
T = [2, 3, 4, 5].

?- [A, B, C|T] = [1, 2, 3, 4, 5].
A = 1,
B = 2,
C = 3,
T = [4, 5].

?- [_|T] = [1, 2, 3, 4, 5].
T = [2, 3, 4, 5]. % _ este discard character, cand nu iti
    pasa de acel parametru
```

Exercitiu de exemplu (P07 din P99):

Se da o lista sub forma `[1, [2, [3, 4], 5], 6]` si trebuie sa o transformati in `[1, 2, 3, 4, 5, 6]`.

Rezolvare:

Listing 1.4: Exercitiu Flatten List

```
my_flatten([], []). % Cazul de baza

% Cazul cand elementul curent este o lista
my_flatten([H|T], R) :-
    is_list(H),
    my_flatten(H, Fh),
    my_flatten(T, Ft),
    append(Fh, Ft, R).

% Cazul cand elementul curent nu este o lista
my_flatten([H|T], [H|Tr]) :-
    \+ is_list(H),  % Operatorul \+ este operatorul not
    my_flatten(T, Tr).
```

Exercitii propuse

Exercitiul 1

Definiti `my_append/3` echivalent cu append-ul built in.

Rulati si confirmati:

```
?- my_append([1, 2, 3], [4, 5, 6], X)
X = [1, 2, 3, 4, 5, 6].
?- my_append([1, 2, X], [4, 5, 6], [1, 2, 3, 4, 5, 6])
X = 3.
?- my_append([X, 2, Z], [4, 5, 6], [1, 2, 3, 4, Y, 6])
X = 1, Z = 3, Y = 5.
?- my_append(L, [4, 5, 6], [1, 2, 3, 4, 5, 6])
L = [1, 2, 3].
?- my_append(L, [4, 5, 6], [1, 2, 3, 5, 6])
false.
```

Exercitiul 2

Definiti `duplicate/2` care repeta de doua ori elementele dintr-o lista.

Rulati si confirmati:

```
?- duplicate([1, 2, 3], X)
X = [1, 1, 2, 2, 3, 3].
?- duplicate([1, 2, X], [1, 1, 2, 2, 3, 3])
X = 3.
?- duplicate([1, 2, X], [1, 1, 2, 2, 3])
false.
```

Exercitiul 3

Definiti `subset/2` care, dat o lista ca primul paremtru, verifica ca al doilea parametru sa fie o submultime a primei liste.

Rulati si confirmati:

```
?- subset([1, 2, 3], X)
X = []; X = [3]; X = [2]; X = [2, 3]; X = [1]; X = [1, 3]; X = [1,
2]; X = [1, 2, 3]
```

Chapter 2

Continuare Prolog, FNC si FND

2.1 Funcții Boolene

Funcțiile boolene sunt o altă metodă de a descrie semantic formule în mod complet (de a le interpreta, cu toate evaluările posibile), fiind echivalente cu tabelele de adevăr. De fapt un tabel de adevăr definește o funcție booleană.

Fie o formulă φ și mulțimea tutoror variabilelor propoziționale din această formulă $Var(\varphi) = \{v_{i_1}, v_{i_2}, \ldots, v_{i_n}\}$. Conform propoziției 3, evaluările diferite ale unei formule depind doar de valorile pe care le iau cele n variabile din formulă, și pot avea, deci 2^n astfel de evaluări. (fiecare variabilă poate fi 0 sau 1). Ceea ce înseamnă că pentru un tabel de adevăr vor fi necesare 2^n intrări.

Este mult mai ușor să codificăm modul de a evalua o formulă printr-o funcție care ia argument n-upluri de 0 sau 1 (fiecare n-uplu reprezintă valorile pe care le iau, în evaluare, variabilele propoziționale) și care dă rezultat în $\{0, 1\}$.

Definition 3. O funcție booleană este o funcție $F : \{0, 1\}^n \to \{0, 1\}$ unde $n \geq 1$.
spunem că n este numărul variabilelor lui F.

Adică, alegem o evaluare $e : V \to \{0, 1\}$, astfel încât:

$$e(v) = \begin{cases} \varepsilon_k, & v \in Var(\varphi), v = v_{i_k} \\ \\ 0 \text{ sau } 1, & \text{altfel} \end{cases}$$

Funcția booleană asociată lui $F_\varphi : \{0, 1\}^n \to \{0, 1\}$ este definită astfel:

$$F_\varphi(\varepsilon_1, \varepsilon_2, \ldots, \varepsilon_n) = e^+(\varphi), \quad \forall\, (\varepsilon_1, \varepsilon_2, \ldots, \varepsilon_n) \in \{0, 1\}^n$$

Chiar dacă, matematic acest lucru pare complicat, nu reprezintă decât formalizarea tabelului de adevăr, unde orice argument de tipul $\varepsilon_1, \varepsilon_2, \ldots, \varepsilon_n$ este un rând din tabelul de adevăr.

De aici apar mai multe consecințe utile

Proposition 5.

(i) Fie φ o formulă. Atunci:

(a) $\models \varphi$ ddacă F_φ este funcția constantă 1 (Adică, în orice evaluare posibilă a variabilelor din formulă) evaluarea formulei va fi mereu adevărată.).

(b) φ este nesatisfiabilă ddacă F_φ este funcția constantă 0.

(ii) Fie φ, ψ două formule astfel încât $\mathrm{Var}(\varphi) = \mathrm{Var}(\psi)$. Atunci:

(a) $\varphi \models \psi$ ddacă $F_\varphi \leq F_\psi$.

(b) $\varphi \sim \psi$ ddacă $F_\varphi = F_\psi$.

(iii) Există formule diferite φ, ψ a.î. $F_\varphi = F_\psi$.

2.2 Forma normală Conjunctivă și Forma Normală Disjunctivă

2.2.1 Concepte Fundamentale

Un concept fundamental în lucrul cu formule este reprezentat de literali.

Definition 4. Un **literal** este fie o variabilă propozițională (caz în care se numește literal pozitiv), fie negația unei variabile (caz în care se numește literal negativ).
Exemple: $v_0, v_{100}, \neg v_{30}, \ldots$

2.2.2 FNC

O formulă φ este în Forma Normală Conjunctivă dacă este o conjuncție ($\wedge$) de disjuncții ($\vee$) de literali.

Un exemplu este: $(v_0 \vee v_1 \vee v_2) \wedge (v_{10} \vee v_5)$.

Mai general, o formulă în FNC are forma

$$\varphi = \bigwedge_{i=1}^{n} (\bigvee_{j=1}^{k_i} (L_{i,j}))$$

Unde $L_{i,j}$ este un literal.

2.2.3 FND

O formulă φ este în Forma Normală Disjunctivă dacă este o disjuncție ($\wedge$) de conjuncții ($\vee$) de literali.

Un exemplu este: $(v_0 \wedge v_1 \wedge v_2) \vee (v_{10} \wedge v_5)$.

Mai general, o formulă în FNC are forma

$$\varphi = \bigvee_{i=1}^{n} (\bigwedge_{j=1}^{k_i} (L_{i,j}))$$

Unde $L_{i,j}$ este un literal.

Note. Există câteva cazuri speciale

- $v_1 \wedge \neg v_5 \wedge v_{10}$ este atât în FNC (conjuncție de literali), cât și în FND (disjuncția cu un singur element e acel element, deci e o disjuncție de o singură conjuncție).
- $\neg v_3 \vee v_4 \vee v_5$ e atât în FNC cât și în FND (ca mai sus)
- v_0 e atât în FNC cât și în FND.

2.2.4 Aducerea formulelor în FNC și FND

Aici puteți să înțelegeți și la ce ne ajută funcțiile boolene: să facem legătura între formule și formele normale.

Theorem 2. Fie $n \geq 1$ și $H : \{0,1\}^n \to \{0,1\}$ o funcție booleană arbitrară. Atunci o există o formulă φ în FNC și o formulă ψ în FND

$$H = F_\varphi = F_\psi$$

Această teoremă este foarte importantă pentru că ne spune următorul lucru: orice formulă φ am defini (și deci orice funcție booleană asociată ei: H), pot găsi o formulă în FNC și o alta în FND care să fie echivalente semantic cu φ (adică să aibă funcțiile boolene egale).

Formal

Theorem 3. Orice formulă φ este echivalentă cu o formulă φ^{FND} în FND și cu o formulă φ^{FNC} în FNC.

2.2.5 Algoritmul de construcție pentru FNC și FND de la un tabel

Există un algoritm mecanic prin care putem extrage FND și FNC pentru orice funcție booleană definită printr-un tabel de adevăr.

Acest algoritm se bazează pe identificarea liniilor din tabel unde funcția se evaluează la 1 (pentru extragerea FND) și la 0 (pentru extragerea FNC).

Proposition 6. Fie $H : \{0,1\}^n \to \{0,1\}$ o funcție booleană.

(i) Fiecărei evaluări $(\varepsilon_1, \ldots, \varepsilon_n)$ pentru care $H(\varepsilon_1, \ldots, \varepsilon_n) = 1$ îi corespunde o conjuncție de literali, definită ca $C = L_1 \wedge L_2 \wedge \cdots \wedge L_n$, unde $L_i = v_i$ dacă $\varepsilon_i = 1$ și $L_i = \neg v_i$ dacă $\varepsilon_i = 0$. Funcția H este echivalentă cu disjuncția tuturor acestor termeni.

(ii) Fiecărei evaluări $(\varepsilon_1, \ldots, \varepsilon_n)$ pentru care $H(\varepsilon_1, \ldots, \varepsilon_n) = 0$ îi corespunde o disjuncție de literali numită, definită ca $D = L_1 \vee L_2 \vee \cdots \vee L_n$, unde $L_i = v_i$ dacă $\varepsilon_i = 0$ și $L_i = \neg v_i$ dacă $\varepsilon_i = 1$. Funcția H este echivalentă cu conjuncția tuturor acestor termeni.

Aplicație pe un exemplu

Exemplu: Fie $H : \{0,1\}^3 \to \{0,1\}$ descrisă prin tabelul:

ε_1	ε_2	ε_3	$H(\varepsilon_1, \varepsilon_2, \varepsilon_3)$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

$D_1 = v_1 \vee v_2 \vee v_3$

$D_2 = v_1 \vee v_2 \vee \neg v_3$

$C_1 = \neg v_1 \wedge v_2 \wedge \neg v_3$

$D_3 = v_1 \vee \neg v_2 \vee \neg v_3$

$C_2 = v_1 \wedge \neg v_2 \wedge \neg v_3$

$C_3 = v_1 \wedge \neg v_2 \wedge v_3$

$C_4 = v_1 \wedge v_2 \wedge \neg v_3$

$C_5 = v_1 \wedge v_2 \wedge v_3$

$\varphi = C_1 \vee C_2 \vee C_3 \vee C_4 \vee C_5$ în FND a.î. $H = F_\varphi$.

$\psi = D_1 \wedge D_2 \wedge D_3$ în FNC a.î. $H = F_\psi$.

2.2.6 Algoritmul de aducere a unei formule la FND/FNC

Aducerea unei formule la Forma Normală Conjunctivă (FNC) sau Forma Normală Disjunctivă (FND) se poate realiza și prin transformări sintactice succesive, folosind echivalențe logice fundamentale, fără a fi necesară construirea întregului tabel de adevăr.
Algoritmul mecanic constă în următorii pași:

Pasul 1. Se înlocuiesc implicațiile și echivalențele, folosind regulile:

$$\varphi \to \psi \sim \neg\varphi \vee \psi \quad \text{și} \quad \varphi \leftrightarrow \psi \sim (\neg\varphi \vee \psi) \wedge (\neg\psi \vee \varphi)$$

Pasul 2. Se înlocuiesc dublele negații, folosind $\neg\neg\psi \sim \psi$, și se aplică regulile lui De Morgan pentru a împinge negațiile în interior, direct lângă variabile (transformându-le în literali):

$$\neg(\varphi \vee \psi) \sim \neg\varphi \wedge \neg\psi \quad \text{și} \quad \neg(\varphi \wedge \psi) \sim \neg\varphi \vee \neg\psi$$

Pasul 3. În funcție de forma dorită, se aplică legile de distributivitate:

- Pentru **FNC**, se aplică distributivitatea disjuncției ($\vee$) față de conjuncție ($\wedge$), pentru a înlocui:

$$\varphi \vee (\psi \wedge \chi) \sim (\varphi \vee \psi) \wedge (\varphi \vee \chi) \quad \text{și} \quad (\psi \wedge \chi) \vee \varphi \sim (\psi \vee \varphi) \wedge (\chi \vee \varphi)$$

- Pentru **FND**, se aplică distributivitatea conjuncției ($\wedge$) față de disjuncție ($\vee$), pentru a înlocui:

$$\varphi \wedge (\psi \vee \chi) \sim (\varphi \wedge \psi) \vee (\varphi \wedge \chi) \quad \text{și} \quad (\psi \vee \chi) \wedge \varphi \sim (\psi \wedge \varphi) \vee (\chi \wedge \varphi)$$

Exemple de aplicare a algoritmului

Exemplul 1 (Aducerea la FNC): Fie formula $\alpha = \neg(p \to q) \vee (r \wedge s)$. Dorim să o aducem la Forma Normală Conjunctivă.

- *Pasul 1 (Eliminarea implicației):*

$$\alpha \sim \neg(\neg p \vee q) \vee (r \wedge s)$$

- *Pasul 2 (Aplicarea legilor De Morgan și eliminarea dublei negații):*

$$\alpha \sim (\neg\neg p \wedge \neg q) \vee (r \wedge s) \sim (p \wedge \neg q) \vee (r \wedge s)$$

- *Pasul 3 (Distributivitatea $\vee$ față de $\wedge$ pentru FNC):* Aplicăm distributivitatea considerând blocul $(p \wedge \neg q)$ ca pe o singură componentă:

$$\alpha \sim ((p \wedge \neg q) \vee r) \wedge ((p \wedge \neg q) \vee s)$$

  Acum aplicăm din nou distributivitatea în interiorul fiecărei paranteze mari:

$$\alpha \sim (p \vee r) \wedge (\neg q \vee r) \wedge (p \vee s) \wedge (\neg q \vee s)$$

Rezultatul final este o conjuncție de disjuncții, formula fiind acum în FNC.

Exemplul 2 (Aducerea la FND): Fie formula $\beta = (p \vee q) \wedge \neg(r \to s)$. Dorim să o aducem la Forma Normală Disjunctivă.

- *Pasul 1 (Eliminarea implicației):*

$$\beta \sim (p \vee q) \wedge \neg(\neg r \vee s)$$

- *Pasul 2 (Aplicarea legilor De Morgan):*

$$\beta \sim (p \vee q) \wedge (\neg\neg r \wedge \neg s) \sim (p \vee q) \wedge (r \wedge \neg s)$$

- *Pasul 3 (Distributivitatea $\wedge$ față de $\vee$ pentru FND):* Aplicăm distributivitatea blocului $(r \wedge \neg s)$ peste disjuncția $(p \vee q)$:

$$\beta \sim (p \wedge (r \wedge \neg s)) \vee (q \wedge (r \wedge \neg s))$$

  Renunțând la parantezele redundante datorită asociativității conjuncției, obținem:

$$\beta \sim (p \wedge r \wedge \neg s) \vee (q \wedge r \wedge \neg s)$$

Rezultatul final este o disjuncție de conjuncții, formula fiind acum în FND.

2.3 Prolog Aritmetica si Exercitii

Aritmetica in Prolog este putin diferita fata de limbajele procedurale si poate fi usor de confundat. Avem urmatorii operatori relevanti: `+, -, *, /, //, is, <, >, >=, =<, =\=, =:=`. Majoritatea sunt intuitivi si fac ceea ce v-ati astepta, dar unele sunt mai confusing si nu fac poate ce credeti ca fac.

Operatorul `is`, utilizat astfel: `X is 2 + 3` sau `X is Y + 1`, va evalua expresia aritmetica din dreapta si va **unifica** rezultatul cu variabila din stanga. Conditie: Toate variabilele din partea dreapta (ex. `Y`) trebuie sa fie **deja instantiate** cu valori numerice.

Operatorul `=:=`, utilizat astfel: `10 =:= 5 + 5`, va evalua expresiile din ambele parti si doar verifica daca rezultatele sunt egale aritmetic. Acesta **nu unifica** variabile; ambele parti trebuie sa poata fi evaluate numeric in momentul apelului.

Similar cu `=:=`, operatorii relationali (`<, >, >=, =<, =\=`) realizeaza comparatii numerice dupa evaluarea ambelor parti si **nu realizeaza unificare**.

Acesti operatori sunt utilizati pentru a restrange universul de solutii, dar nu pot fi folositi pentru a "atribui" valori variabilelor, cu exceptia lui `is`.

Exemple relevante:

```
?- X is 5 + 2.
X = 7.
?- 7 is 5 + 2.
true.
?- 10 is 5 + 2.
false.
?- 10 =:= 5 + 5.
true.
?- X =:= 5 + 2.
ERROR: Arguments are not sufficiently instantiated.
```

Exercitii rezolvate

Note (Sfaturi generale)**.** In rezolvari de obicei e util sa ne gandim la toate cazurile de baza/triviale. Puteti sa scrieti oricate cazuri va vin in minte la inceput ca sa intelegeti problema mai bine, chiar daca sunt acoperite de cazurile recursive (recomandat e sa le stergeti dupa ca posibil sa interfereze in unele cazuri).

In unele cazuri unele probleme ies mult mai usor daca va definiti predicate auxiliare/intermediare (pot avea chiar si acelasi nume atat timp cat au un numar diferit de parametrii). De exemplu:

Listing 2.1: Auxiliary predicates example

```
square(0, _).
square(W, C) :- square(W, W, C).

square(0, _, _).
square(I, W, C) :-
    I > 0,
    I1 is I - 1,
    square(I1, W, C),
    squareRow(W, C).

squareRow(0, _) :- nl.
squareRow(N, C) :-
    N > 0,
    N1 is N - 1,
    squareRow(N1, C),
    write(C).
```

Exercitiul 1

Definiti `suma_lista/2` care face suma elementelor din lista data ca prim argument si pune rezultatul in al doilea argument.

Rulati si confirmati:

```
?- suma_lista([1, 2, 3], 6).
true.
?- suma_lista([1, 2, 3], X).
X = 6.
?- suma_lista([1, 2, X], 6).
ERROR: Arguments are not sufficiently instantiated.
```

Solutie:

```
suma_lista([], 0).
suma_lista([H|T], Sr) :-
    suma_lista(T, S),
    Sr is H + S.
```

Exercitiul 2

Definiti `medie/2` care face media aritmetica a elementelor din lista data ca prim argument si pune rezultatul in al doilea argument.

Rulati si confirmati:

```
?- medie([1, 2, 3, 4], X).
X = 2.5 .
?- medie([1, 2, 3, X], 2.5).
ERROR: Arguments are not sufficiently instantiated.
```

Solutie:

```
% media([], _) :- false.
% media([A], A).
media(L, R) :-
    stats(L, S, N),
    N > 0,
    R is S / N.

stats([], 0, 0).
stats([H|T], S, N) :-
    stats(T, S_coada, N_coada),
    S is H + S_coada,
    N is N_coada + 1.
```

Exercitii propuse

> **Remark.** Incercati sa 'suciti' apelurile in toate directiile cand testati ca sa vedeti cand reuseste si cand nu reuseste sa unifice si sa gaseasca un rezultat (si daca credeti ca ar trebui sa poata, incercati sa solutionati).

Exercitiul 1

Definiti `split/4` care sparge lista primita ca prim argument la pozitia primita ca al doilea argument in doua liste (parametrii 3 si 4).

Rulati si confirmati:

```
?- split([1, 2, 3, 4], 2, [1, 2], [3, 4]).
true.
?- split([1, 2, 3, 4], 2, A, B).
A = [1, 2], B = [3, 4].
?- split(X, 2, [1, 2], [3, 4]).
X = [1, 2, 3, 4].
?- split([1, 2, 3], 0, [], [1, 2, 3]).
true.
?- split([1, 2, 3], X, [1], [2, 3]).
ERROR: Arguments not sufficiently instantiated.
```

Exercitiul 2

Definiti predicatul `filtreaza_peste/3` care extrage numerele dintr-o lista (primul parametru) care sunt peste un anumit prag (al doilea parametru) intr-o alta lista (al treilea parametru).

Rulati si confirmati:

```
?- filtreaza_peste([1, 5, 10, 2, 8], 5, R).
R = [10, 8].
?- filtreaza_peste([10, 20, 30], 40, R).
R = [].
```

Exercitiul 3

Definiti predicatul `intsum/3` care calculeaza suma tuturor numerelor intregi cuprinse intr-un interval [a, b] (primele doua argumente) si pune rezultatul in ultimul argument.

Rulati si confirmati:

```
?- intsum(1, 4, S).
S = 10.
```
$(1 + 2 + 3 + 4 = 10)$

```
?- intsum(5, 5, S).
S = 5.
```

Exercitiul 4

Definiti predicatul `putere/3` care ridica numarul primit ca prim parametru la puterea data ca al doilea parametru si pune rezultatul in ultimul parametru.

Rulati si confirmati:

```
?- putere(3, 3, X).
X = 27.
?- putere(5, 0, X).
X = 1.
?- putere(5, 2, X).
X = 25.
```

Chapter 3

Clauze, Rezoluție și Prolog

3.1 Continuare FNC și FND

Exercițiile de la forme normale sunt foarte algoritmice și există foarte mare șansă să aveți la examen acest tip de exercițiu. Așa că e bine să vă obișnuiți să lucrați cu ele. Există două tipuri de exerciții: transformarea unei formule în FNC/FND prin transformare sintactică (adică al doilea algoritm prezentat în capitolul anterior) și prin aducerea la funcție booleană (adică compunerea tabelului de adevăr a formulului și aplicarea primului algoritm din capitolul anterior).

Exercițiu. Să se aducă următoarele formule la cele două forme normale prin transformări sintactice:

(i) $((v_0 \to v_1) \wedge v_1) \to v_0$;

(ii) $(v_1 \vee \neg v_4) \to (\neg v_2 \to v_3)$.

Soluție. Vom folosi algoritmul detaliat în 2.2.6.

Pentru formula (i): $\alpha = ((v_0 \to v_1) \wedge v_1) \to v_0$

- *Pasul 1 (Eliminarea implicațiilor):* Aplicăm regula $\varphi \to \psi \sim \neg\varphi \vee \psi$:

$$\alpha \sim \neg((\neg v_0 \vee v_1) \wedge v_1) \vee v_0$$

- *Pasul 2 (Aplicarea legilor De Morgan și absorbția):* Introducem negația în paranteză folosind De Morgan:

$$\alpha \sim (\neg(\neg v_0 \vee v_1) \vee \neg v_1) \vee v_0$$

  Aplicăm din nou De Morgan pentru termenul $\neg(\neg v_0 \vee v_1)$:

$$\alpha \sim ((v_0 \wedge \neg v_1) \vee \neg v_1) \vee v_0$$

- *Pasul 3 (Aplicarea distributivității și a idempotenței):* Aplicăm distributivitatea disjuncției față de conjuncție pentru subformula $((v_0 \wedge \neg v_1) \vee \neg v_1)$:

$$\alpha \sim ((v_0 \vee \neg v_1) \wedge (\neg v_1 \vee \neg v_1)) \vee v_0$$

Aplicăm legea idempotenței pentru disjuncție ($\neg v_1 \vee \neg v_1 \sim \neg v_1$):

$$\alpha \sim ((v_0 \vee \neg v_1) \wedge \neg v_1) \vee v_0$$

- *Pasul 4 (A doua aplicare a distributivității):* Pentru a finaliza aducerea la forma normală, distribuim disjuncția ($\vee v_0$) peste conjuncția rămasă:

$$\alpha \sim ((v_0 \vee \neg v_1) \vee v_0) \wedge (\neg v_1 \vee v_0)$$

Folosind comutativitatea, asociativitatea și idempotența ($v_0 \vee v_0 \sim v_0$) în prima paranteză, obținem:

$$\alpha \sim (v_0 \vee \neg v_1) \wedge (v_0 \vee \neg v_1)$$

În final, aplicăm idempotența pentru întreaga expresie ($A \wedge A \sim A$), rezultând o formulă care e atât în FNC, cât și în FND:

$$\alpha \sim v_0 \vee \neg v_1$$

Pentru formula (ii): $\beta = (v_1 \vee \neg v_4) \rightarrow (\neg v_2 \rightarrow v_3)$

- *Pasul 1 (Eliminarea implicațiilor):* Mai întâi eliminăm implicația din paranteza dreaptă:

$$\beta \sim (v_1 \vee \neg v_4) \rightarrow (\neg\neg v_2 \vee v_3) \sim (v_1 \vee \neg v_4) \rightarrow (v_2 \vee v_3)$$

Apoi eliminăm implicația principală:

$$\beta \sim \neg(v_1 \vee \neg v_4) \vee (v_2 \vee v_3)$$

- *Pasul 2 (Aplicarea legilor De Morgan):*

$$\beta \sim (\neg v_1 \wedge \neg\neg v_4) \vee v_2 \vee v_3 \sim (\neg v_1 \wedge v_4) \vee v_2 \vee v_3$$

În acest stadiu, formula este deja o disjuncție de conjuncții, așadar avem forma FND.

- *Pasul 3 (Aducerea la FNC prin distributivitate):* Pentru a obține FNC, distribuim disjuncția ($\vee v_2 \vee v_3$) peste conjuncția ($\neg v_1 \wedge v_4$):

$$\beta \sim (\neg v_1 \vee v_2 \vee v_3) \wedge (v_4 \vee v_2 \vee v_3)$$

□

Exercițiu. Să se aducă formula $\varphi = (v_0 \to v_1) \to v_2$ la cele două forme normale trecându-se prin funcția booleană asociată (i.e. metoda tabelului).

Soluție. Construim tabelul de adevăr pentru funcția booleană asociată formulei φ. Variabilele sunt v_0, v_1, v_2.

v_0	v_1	v_2	$v_0 \to v_1$	φ
0	0	0	1	0
0	0	1	1	1
0	1	0	1	0
0	1	1	1	1
1	0	0	0	1
1	0	1	0	1
1	1	0	1	0
1	1	1	1	1

$D_1 = v_0 \vee v_1 \vee v_2$
$C_1 = \neg v_0 \wedge \neg v_1 \wedge v_2$
$D_2 = v_0 \vee \neg v_1 \vee v_2$
$C_2 = \neg v_0 \wedge v_1 \wedge v_2$
$C_3 = v_0 \wedge \neg v_1 \wedge \neg v_2$
$C_4 = v_0 \wedge \neg v_1 \wedge v_2$
$D_3 = \neg v_0 \vee \neg v_1 \vee v_2$
$C_5 = v_0 \wedge v_1 \wedge v_2$

Extragem **Forma Normală Disjunctivă (FND)** prin disjuncția conjuncțiilor C_i corespunzătoare liniilor unde $\varphi = 1$:

$\varphi^{FND} = C_1 \vee C_2 \vee C_3 \vee C_4 \vee C_5$
$\varphi^{FND} = (\neg v_0 \wedge \neg v_1 \wedge v_2) \vee (\neg v_0 \wedge v_1 \wedge v_2) \vee (v_0 \wedge \neg v_1 \wedge \neg v_2) \vee (v_0 \wedge \neg v_1 \wedge v_2) \vee (v_0 \wedge v_1 \wedge v_2)$

Extragem **Forma Normală Conjunctivă (FNC)** prin conjuncția disjuncțiilor D_i corespunzătoare liniilor unde $\varphi = 0$ (se neagă valoarea de adevăr a variabilelor de pe linie):

$\varphi^{FNC} = D_1 \wedge D_2 \wedge D_3$
$\varphi^{FNC} = (v_0 \vee v_1 \vee v_2) \wedge (v_0 \vee \neg v_1 \vee v_2) \wedge (\neg v_0 \vee \neg v_1 \vee v_2)$

□

3.2 Clauze și forma clauzală

Până acum am lucrat cu formule în FNC/FND. Pentru algoritmi de satisfiabilitate și pentru rezoluție, este util să tratăm formulele în FNC drept **mulțimi de clauze**. Asta simplifică mult calculele, pentru că ordinea literalilor nu contează, iar duplicatele se pot elimina imediat.

3.2.1 Definiții de bază

Definition 5. O **clauză** este o mulțime finită de literali:

$$C = \{L_1, L_2, \ldots, L_n\}.$$

Dacă $n = 0$, obținem **clauza vidă**, notată □, adică $\square = \emptyset$.

Dacă L este un literal, notăm literalul complementar cu L^c:

$$L^c = \begin{cases} \neg v, & \text{dacă } L = v \\ v, & \text{dacă } L = \neg v \end{cases}$$

Intuitiv, o clauză nevidă se citește ca o disjuncție de literali. De exemplu:

$$\{v_1, \neg v_3, v_5\} \quad \leftrightarrow \quad v_1 \vee \neg v_3 \vee v_5.$$

Definition 6. Fie C o clauză și $e : V \to \{0, 1\}$ o evaluare. Spunem că e satisface clauza C, și scriem $e \models C$, dacă există un literal $L \in C$ cu $e \models L$. Pentru o mulțime finită de clauze $S = \{C_1, \ldots, C_m\}$, scriem $e \models S$ dacă $e \models C_i$ pentru orice i. Deci S se citește ca o conjuncție de clauze.

Definition 7. Fie C o clauză și S o mulțime finită de clauze.

1. Spunem că C este **satisfiabilă** dacă există o evaluare e cu $e \models C$. Spunem că C este **validă** dacă pentru orice evaluare e, avem $e \models C$.
2. Spunem că S este **satisfiabilă** dacă există o evaluare e cu $e \models S$. Spunem că S este **validă** dacă pentru orice evaluare e, avem $e \models S$.

Proposition 7.

1. Orice clauză nevidă este satisfiabilă.
2. Clauza vidă $\square$ este nesatisfiabilă.
3. Dacă o clauză conține simultan L și L^c, atunci este validă (trivială).

3.2.2 Legătura dintre clauze și FNC

Pentru o formulă în FNC

$$\varphi = \bigwedge_{i=1}^{n} \left(\bigvee_{j=1}^{k_i} L_{i,j} \right),$$

asociem forma clauzală

$$S_\varphi = \{C_1, \ldots, C_n\}, \quad C_i = \{L_{i,1}, \ldots, L_{i,k_i}\},$$

unde într-o clauză păstrăm fiecare literal o singură dată.

Exercițiu. Fie

$$\varphi = (v_1 \vee \neg v_2 \vee v_1) \wedge (\neg v_1 \vee v_3) \wedge (v_2).$$

Să se scrie forma clauzală a formulei.

Soluție. Transformăm fiecare paranteză într-o clauză și eliminăm duplicatele:

$$S_\varphi = \{\{v_1, \neg v_2\}, \{\neg v_1, v_3\}, \{v_2\}\}.$$

Observația importantă este că φ și S_φ sunt semantic echivalente: au exact aceleași modele. $\square$

3.2.3 Conversia inversă: din clauze la formula φ_S

Conversia se face în doi pași: mai întâi transformăm fiecare clauză într-o disjuncție, apoi facem conjuncția tuturor acestor formule.

Definition 8. Pentru o clauză C, formula asociată φ_C este:

1. dacă $C = \{L_1, \ldots, L_n\}$, cu $n \geq 1$, atunci
$$\varphi_C := L_1 \vee \cdots \vee L_n;$$
2. dacă $C = \square$, atunci putem lua
$$\varphi_\square := v_0 \wedge \neg v_0.$$

Definition 9. Pentru o mulțime finită de clauze S, formula asociată φ_S este:

1. dacă $S = \{C_1, \ldots, C_m\}$, cu $m \geq 1$, atunci
$$\varphi_S := \bigwedge_{i=1}^{m} \varphi_{C_i};$$
2. dacă $S = \emptyset$, atunci putem lua
$$\varphi_\emptyset := v_0 \vee \neg v_0.$$

Note. Formula φ_S nu este unică sintactic (ordinea literalilor/clauzelor poate diferi), dar toate variantele sunt echivalente semantic.

Proposition 8. Pentru orice evaluare e, avem:

$$e \models S \quad \text{ddacă} \quad e \models \varphi_S.$$

Exercițiu. Fie

$$S = \{\{v_1, \neg v_2\}, \{v_3\}, \{\neg v_1, v_4\}\}.$$

Scrieți formula φ_S.

Soluție. Asociem formulele pe clauze:

$$\varphi_{C_1} = v_1 \vee \neg v_2, \quad \varphi_{C_2} = v_3, \quad \varphi_{C_3} = \neg v_1 \vee v_4.$$

Deci:

$$\varphi_S = (v_1 \vee \neg v_2) \wedge v_3 \wedge (\neg v_1 \vee v_4).$$

□

3.3 Rezoluția pe clauze

Rezoluția este o regulă de inferență specială pentru clauze. Ea este folosită ca să verificăm dacă un set de condiții este compatibil sau nu. Pe scurt, dacă două clauze se „bat cap în cap" pe aceeași variabilă (una conține L, cealaltă conține L^c), eliminăm perechea contradictorie și păstrăm ce rămâne.

Exemplu:

$$\{p, q\} \text{ și } \{\neg p, r\} \implies \{q, r\}.$$

Am eliminat p și $\neg p$, pentru că sunt complementare.

Dacă, după mai mulți astfel de pași, ajungem la clauza vidă □, înseamnă că am obținut contradicție, deci clauzele inițiale nu pot fi satisfăcute împreună. Exact de aceea rezoluția este utilă în verificare automată, în solvere SAT și în mecanismul logic din PROLOG.

Definition 10. Fie C_1, C_2 două clauze. O clauză R se numește **rezolvent** al lor dacă există un literal L astfel încât

$$L \in C_1, \quad L^c \in C_2, \quad R = (C_1 \setminus \{L\}) \cup (C_2 \setminus \{L^c\}).$$

Exercițiu. Fie $C_1 = \{v_1, \neg v_2, v_4\}$ și $C_2 = \{v_2, v_3\}$. Să se calculeze un rezolvent.

Soluție. Alegem literalul $L = \neg v_2$, deci $L^c = v_2$.

$$R = (C_1 \setminus \{\neg v_2\}) \cup (C_2 \setminus \{v_2\}) = \{v_1, v_4, v_3\}.$$

Acesta este un rezolvent al lui C_1 și C_2. □

Proposition 9 (Corectitudinea rezoluției)**.** Dacă dintr-o mulțime de clauze S se poate deriva clauza vidă □ prin pași de rezoluție, atunci S este nesatisfiabilă.

Exercițiu. Arătați prin rezoluție că

$$S = \{\{v_1\}, \{\neg v_1, v_2\}, \{\neg v_2\}\}$$

este nesatisfiabilă.

Soluție. Notăm

$$C_1 = \{v_1\}, \quad C_2 = \{\neg v_1, v_2\}, \quad C_3 = \{\neg v_2\}.$$

Rezolvăm C_1 cu C_2 după literalul v_1:

$$C_4 = \{v_2\}.$$

Rezolvăm C_4 cu C_3 după literalul v_2:

$$C_5 = \square.$$

Am derivat clauza vidă, deci S este nesatisfiabilă. $\square$

3.4 Algoritmul Davis–Putnam (DP)

Algoritmul Davis–Putnam decide satisfiabilitatea unei mulțimi finite de clauze prin **eliminare de variabile** folosind rezolvenți.

Algoritmul DP (scriere standard)

Intrare: S mulțime finită nevidă de clauze netriviale.

$$i := 1, \quad S_1 := S.$$

Pi.1 Fie x_i o variabilă care apare în S_i. Definim:

$$T_i^1 := \{C \in S_i \mid x_i \in C\}, \qquad T_i^0 := \{C \in S_i \mid \neg x_i \in C\}.$$

Pi.2 Dacă $T_i^1 \neq \emptyset$ și $T_i^0 \neq \emptyset$, atunci

$$U_i := \{(C_1 \setminus \{x_i\}) \cup (C_0 \setminus \{\neg x_i\}) \mid C_1 \in T_i^1,\ C_0 \in T_i^0\}.$$

altfel $U_i := \emptyset$.
Pi.3 Definim:

$$S'_{i+1} := (S_i \setminus (T_i^0 \cup T_i^1)) \cup U_i,$$
$$S_{i+1} := S'_{i+1} \setminus \{C \in S'_{i+1} \mid C \text{ trivială}\}.$$

Pi.4 Facem testul de oprire:

dacă $S_{i+1} = \emptyset$ atunci S este satisfiabilă;

altfel dacă $\square \in S_{i+1}$ atunci S este nesatisfiabilă;

altfel $i := i + 1$ și revenim la Pi.1.

Explicația pașilor

Intuitiv, la fiecare iterație alegem o variabilă x_i și „o eliminăm” din sistem, păstrând doar informația relevantă pentru satisfiabilitate.
Pasul Pi.1. Împărțim clauzele din S_i în două grupuri: cele care conțin x_i (acestea formează T_i^1) și cele care conțin $\neg x_i$ (acestea formează T_i^0). Clauzele care nu conțin nici x_i, nici $\neg x_i$, rămân pe loc.
Pasul Pi.2. Dacă avem clauze în ambele grupuri (T_i^1 și T_i^0), construim toate rezolventele posibile între ele. Mulțimea acestor rezolvente este U_i. Dacă unul dintre grupuri este vid, nu avem ce rezolva și luăm $U_i = \emptyset$.
Pasul Pi.3. Eliminăm din S_i toate clauzele care conțin x_i sau $\neg x_i$, apoi adăugăm rezolventele din U_i. După aceea, eliminăm clauzele triviale (cele care conțin simultan un literal și complementul său), deoarece nu ajută la decizie.
Pasul Pi.4. Dacă după această simplificare obținem mulțimea vidă de clauze, problema este satisfiabilă. Dacă apare clauza vidă $\square$, problema este nesatisfiabilă. În rest, repetăm aceeași procedură cu o nouă variabilă.

Exercițiu. Fie

$$S = \{\{v_1, \neg v_3\}, \{v_2, v_1\}, \{v_2, \neg v_1, v_3\}\}.$$

Să se aplice DP.

Soluție. $S = \{\{v_1, \neg v_3\}, \{v_2, v_1\}, \{v_2, \neg v_1, v_3\}\}$, $i := 1$, $S_1 := S$.
P1.1 $x_1 := v_3$, $T_1^1 := \{\{v_2, \neg v_1, v_3\}\}$, $T_1^0 := \{\{v_1, \neg v_3\}\}$.
P1.2 $U_1 := \{\{v_2, \neg v_1, v_1\}\}$.
P1.3 $S_2' := \{\{v_2, v_1\}, \{v_2, \neg v_1, v_1\}\}$, $S_2 := \{\{v_2, v_1\}\}$.
P1.4 $i := 2$ and go to P2.1.

P2.1 $x_2 := v_2$, $T_2^1 := \{\{v_2, v_1\}\}$, $T_2^0 := \emptyset$.
P2.2 $U_2 := \emptyset$.
P2.3 $S_3 := \emptyset$.
P2.4 S este satisfiabilă. $\square$

Exercițiu. Fie

$$S = \{\{\neg v_1, v_2, \neg v_4\}, \{\neg v_3, \neg v_2\}, \{v_1, v_3\}, \{v_1\}, \{v_3\}, \{v_4\}\}.$$

Să se aplice DP și să se decidă satisfiabilitatea.

Soluție. $S = \{\{\neg v_1, v_2, \neg v_4\}, \{\neg v_3, \neg v_2\}, \{v_1, v_3\}, \{v_1\}, \{v_3\}, \{v_4\}\}$,
$i := 1$, $S_1 := S$.
P1.1 $x_1 := v_1$, $T_1^1 := \{\{v_1, v_3\}, \{v_1\}\}$, $T_1^0 := \{\{\neg v_1, v_2, \neg v_4\}\}$.
P1.2 $U_1 := \{\{v_3, v_2, \neg v_4\}, \{v_2, \neg v_4\}\}$.
P1.3 $S_2 := \{\{\neg v_3, \neg v_2\}, \{v_3\}, \{v_4\}, \{v_3, v_2, \neg v_4\}, \{v_2, \neg v_4\}\}$.
P1.4 $i := 2$ and go to P2.1.

P2.1 $x_2 := v_2$, $T_2^1 := \{\{v_3, v_2, \neg v_4\}, \{v_2, \neg v_4\}\}$,
$T_2^0 := \{\{\neg v_3, \neg v_2\}\}$.
P2.2 $U_2 := \{\{v_3, \neg v_4, \neg v_3\}, \{\neg v_4, \neg v_3\}\}$.

P2.3 $S_3 := \{\{v_3\}, \{v_4\}, \{\neg v_4, \neg v_3\}\}$.
P2.4 $i := 3$ and go to P3.1.

P3.1 $x_3 := v_3$, $T_3^1 := \{\{v_3\}\}$, $T_3^0 := \{\{\neg v_4, \neg v_3\}\}$.
P3.2 $U_3 := \{\{\neg v_4\}\}$.
P3.3 $S_4 := \{\{v_4\}, \{\neg v_4\}\}$.
P3.4 $i := 4$ and go to P4.1.

P4.1 $x_4 := v_4$, $T_4^1 := \{\{v_4\}\}$, $T_4^0 := \{\{\neg v_4\}\}$.
P4.2 $U_4 := \{\square\}$.
P4.3 $S_5 := \{\square\}$.
P4.4 S este nesatisfiabilă. $\square$

3.5 Prolog findall/setof/bagof

In prolog, in mod normal, cand pui un query, sistemul iti returneaza prima solutie si continua cautarea pentru mai multe solutii doar daca vrem noi.

Predicatele `findall`, `bagof`, `setof` sunt folosite atunci cand vrem sa colectam **toate** solutiile intr-o lista.

Diferentele intre ele stau in modul in care trateaza duplicatele, sortarea, lipsa solutiilor si, cel mai important, variabilele libere.

Vom analiza cum se comporta fiecare incepand cu urmatoarele fapte simple:

```
angajat(dana, it).
angajat(alex, it).
angajat(maria, hr).
angajat(alex, it). % Duplicat intentionat
```

findall(sablon, scop, listaRezultat)

Este cel mai simplu si intuitiv si cel mai des folosit in practica. Face exact ce spune: gaseste toate solutiile si le pune intr-o lista.

Pastreaza duplicate, nu sorteaza si **ignora variabilele libere**.

Cand nu gaseste nimic returneaza o lista goala (nu da false).

Exemple:

```
?- findall(Nume, angajat(Nume, pr), R).
R = [].

?- findall(Nume, angajat(Nume, Dept), R).
R = [dana, alex, maria, alex].
```

Se observa cum a ignorat complet variabila libera Dept, a pastrat duplicatul 'alex' si nu a sortat nimic.

bagof(sablon, scop, listaRezultat)

Predicatul `bagof` este putin mai inteligent in sensul ca **grupeaza** rezultatele in functie de variabilele libere (variabilele care sunt in `scop` dar nu sunt in `sablon`).

Pastreaza duplicate, nu sorteaza, si daca exista variabile libere, Prolog **va returna mai multe liste**, cate una pentru fiecare valoare posibila a variabilei libere.

Cand nu gaseste nimic **esueaza** (returneaza false).

Exemple:

```
?- bagof(Nume, angajat(Nume, pr), R).
false.

?- bagof(Nume, angajat(Nume, Dept), R).
Dept = hr,
R = [maria] ;
Dept = it,
R = [dana, alex, alex].
```

De aici putem sa incepem sa si combinam, de exemplu daca vrem o lista de tupluri de tipul:

`[(numeDepartament, [angajati in departament])]`

Putem face:

```
?- findall((Dept, L), bagof(Nume, angajat(Nume, Dept), L), R).
R = [(hr, [maria]), (it, [dana, alex, alex])].
```

setof(sablon, scop, listaRezultat)

Predicatul `setof` este cel mai strict. El modeleaza o multime matematica (un set), si, in general, elementele unei multimi sunt ordonate.

Elimina duplicate, **sorteaza** elementele (alfabetic sau numeric), si similar cu bagof, grupeaza dupa variabilele libere si esueaza cand nu gaseste nimic

Exemple:

```
?- setof(Nume, angajat(Nume, pr), R).
false.

?- setof(Nume, angajat(Nume, Dept), R).
Dept = hr,
R = [maria] ;
Dept = it,
R = [alex, dana].
```

Se observa cum a eliminat duplicatul 'alex' si a sortat listele.

```
?- findall((Dept, L), setof(Nume, angajat(Nume, Dept), L), R).
R = [(hr, [maria]), (it, [alex, dana])].
```

Operatorul ^

Operatorul ^ este folosit in `setof` si `bagof` sa ignore o variabila libera data (similar cu `findall` dar putem controla ce variabile libere sunt ignorate).

Exemple:

```
?- setof(Nume, Dept^angajat(Nume, Dept), R).
R = [alex, dana, maria].
```

In cazul in care avem mai multe variabile libere pe care vrem sa le ignoram putem face:

```
?- setof(Nume, Firma^Dept^lucreaza(Nume, Oras, Firma, Dept), R).
```

Exercitii rezolvate

Exercitiul 1

Avand ca si context urmatoarele fapte:

```
% inventar(Jucator, Obiect, ValoareAur).
inventar(arthas, sabie, 150).
inventar(arthas, potiune_viata, 10).
inventar(arthas, scut, 50).
inventar(jaina, toiag, 200).
inventar(jaina, potiune_mana, 15).
inventar(jaina, potiune_mana, 15). % Are doua potiuni
```

Definiti predicatul `avereJucator/2` care primeste un jucator ca prim parametru si in al doilea parametru se afla aurul total pe care il are acel jucator.

Rulati si confirmati:

```
?- avere_jucator(arthas, R).
R = 210.
?- avere_jucator(jaina, R).
R = 230.
```

Atentie! Ce credeti ca se intampla daca rulam:

```
?- avere_jucator(X, 210).
?- avere_jucator(X, 440).
?- avere_jucator(X, R).
```

Solutii:

Solutia 1 findall:

```
avere_jucator(J, R) :-
    findall(V, inventar(J, _, V), L),
    suma_lista(L, R).
```

In aceasta solutie daca rulam:

```
?- avere_jucator(X, 210).
false.
?- avere_jucator(X, 440).
true.
?- avere_jucator(X, R).
R = 440.
```

De ce? Pentru ca, daca ii dam sa caute jucatorul, atunci J o sa fie o variabila libera. In `findall`, variabilele libere sunt ignorate. Prin urmare, linia cu `findall(V, inventar(J, _, V), L)`, va returna mereu lista cu toate valorile: `[150, 10, 50, 200, 15, 15]` si suma va fi 440.

Solutia 2 bagof:

```
avere_jucator(J, R) :-
    bagof(V, Obj^inventar(J, Obj, V), L),
    suma_lista(L, R).
```

In aceasta solutie daca rulam:

```
?- avere_jucator(X, 210).
X = arthas.
?- avere_jucator(X, 440).
false.
?- avere_jucator(X, R).
X = arthas,
R = 210 ;
X = jaina,
R = 230.
```

De ce? Pentru ca `bagof` va grupa dupa J cand este variabila libera si cand verifica va lua toate gruparile posibile si forma listele: `(arthas, [150, 10, 50])` si `(jaina, [200, 15, 15])` si va lua jucatorul care are suma listei egala cu 210.

Exercitiul 2

Avand ca si context urmatoarele fapte:

```
% mina(X, Y).
mina(1, 1).
mina(1, 3).
mina(2, 2).
mina(3, 1).
```

Definiti predicatul `mine_vecine/3` care primeste coordonatele X si Y (primii doi parametrii) si returneaza numarul de mine vecine prin al treilea parametru.

Rulati si confirmati:

```
?- mine_vecine(1, 2, R).
R = 3.
?- mine_vecine(1, 4, R).
R = 1.
?- mine_vecine(1, 5, R).
R = 0.
```

Solutie:

```
vecin(X1,Y1, X2,Y2) :-
    between(-1, 1, Dx),
    between(-1, 1, Dy),
    \+ (Dx = 0, Dy = 0), % Sau (Dx =\= 0; Dy =\= 0)
    X2 is X1 + Dx,
    Y2 is Y1 + Dy.

mine_vecine(X, Y, R) :-
    findall((Vx,Vy), (vecin(X,Y, Vx,Vy), mina(Vx,Vy)), L),
    length(L, R).
```

Exercitii propuse:

Exercitiul 1

Definiti predicatul `genAllBetween/3` care genereaza toate numerele dintre A si B (primii doi parametrii) si pune rezultatul ca o lista in ultimul parametru. (Hint: folositi `between/3` si `findall/3`)

Rulati si confirmati:

```
?-genAllBetween(4, 10, R).
R = [4, 5, 6, 7, 8, 9, 10]
?-genAllBetween(4, 4, R).
R = [4]
?-genAllBetween(5, 4, R).
R = []
```

Exercitiul 2

Definiti predicatul `aboveMean/2` care gaseste elementele mai mari decat media tuturor elementelor din prima lista si le pune in a doua lista. Implementati folosind una dintre predicatele `findall/3`, `bagof/3` sau `setof/3`

Rulati si confirmati:

```
?-aboveMean([3, 7, 2, 8, 5, 10], R).
R = [7, 8, 10].
?-aboveMean([1, 1, 1, 1, 1, 10], R).
R = [10].
```

Exercitiul 3

Avand ca si context urmatoarele fapte:

```
% nota(Student, Materie, Nota).
nota(andrei, mate1, 8).
nota(andrei, fizica, 4).
nota(andrei, programare1, 10).
nota(maria, mate1, 6).
nota(maria, fizica, 7).
nota(maria, programare1, 10).

% cerinta(MaterieDorita, MaterieCeTrebuiePromovata).
cerinta(mate2, mate1).
cerinta(programare2, programare1).
cerinta(robotica, programare1).
cerinta(robotica, fizica). % Robotica are doua cerinte
```

Definiti predicatul `meterii_eligibile/2` care primeste numele unui student ca primul parametru si in al doilea parametru se afla lista materiilor la care poate sa se inscrie.

Rulati si confirmati:

```
?- materii_eligibile(andrei, X).
X = [mate2, programare2].
?- materii_eligibile(maria, X).
X = [mate2, programare2, robotica].
```

Chapter 4

Sintaxa LP. Rezoluție și Prolog

4.1 Termeni compusi in Prolog

Pana acum in prolog, am lucrat doar cu termeni simpli (variabile, constante, atomi), dar in prolog mai exista si conceptul de **termeni compusi**.

De exemplu putem codifica formula logica $\varphi = (v_0 \vee v_1) \rightarrow v_2$ astfel:

`implica(sau(v0, v1), v2)`

In acest exemplu, `implica` **nu este un fapt** (nu avem punct la final) ci un termen compus.

Ca sa intelegem cum se lucreaza cu ele, putem lua urmatorul exemplu unde vrem sa aflam toate variabilele unei expresii logice (pentru φ definit anterior variabilele ar fi `[v0, v1, v2]`). Pentru simplitate vom afisa si duplicate.

```
vars(A, [A]) :- atom(A).

vars(not(A), R) :- vars(A, R).

vars(si(A, B), R) :-
    vars(A, R1),
    vars(B, R2),
    append(R1, R2, R).

vars(sau(A, B), R) :-
    vars(A, R1),
    vars(B, R2),
    append(R1, R2, R).

vars(implica(A, B), R) :-
    vars(A, R1),
    vars(B, R2),
    append(R1, R2, R).
```

```
?- vars(implica(sau(v0, v1), v2), R).
R = [v0, v1, v2].
```

```
?- vars(sau(si(v1, v2), si(v2, v4)), R).
R = [v1, v2, v2, v4].
```

Predicate utile

Predicatul `functor(termen compus, functor, aritate)` extrage functorul si aritatea termenului compus:

```
?- functor(data(22, aprilie, 2026), F, A).
F = data,
A = 3.
```

Predicatul `arg(i, termen compus, rez)` extrage termenul de la pozitia i din termenul compus (atentie, este indexat de la 1):

```
?- arg(2, data(22, aprilie, 2026), Luna).
Luna = aprilie.
```

Operatorul `=..` transforma un termen compus intr-o lista si invers:

```
?- data(22, aprilie, 2026) =..  Lista.
Lista = [data, 22, aprilie, 2026].
?- Data =..  [data, 22, aprilie, 2026].
Data = data(22, aprilie, 2026).
```

Exercitii propuse:

Exercitiul 1

Definiti predicatul `suma_noduri(Arbore, Suma)` care calculeaza suma nodurilor unui arbore binar. Folositi termenul compus `nod(Valoare, ArbStang, ArbDrept)`.

```
?- suma_noduri(nod(5, nod(3, nod(1, nil, nil), nod(4, nil, nil)),
nod(8, nil, nil)), R).
R = 21.
```

Exercitiul 2

Definiti predicatul `substituie(ExpresieInitiala, CeInlocuiesc, CuCeInlocuiesc, ExpresieFinala)` care sparge recursiv o structura si inlocuieste aparitiile unui subtermen specific cu altul.

```
?- substituie(implica(v1, si(v1, v3)), v1, not(v2), R).
R = implica(not(v2), si(not(v2), v3)).
```

Exercitiul 3

Definiti predicatul `evaluare(Expresie, ListaAdevar, Rezultat)` care primeste o expresie logica si o lista de variabile care sunt considerate adevarate.

Rulati si confirmati:

```
?- evaluare(si(v1, sau(v2, v3)), [v1, v3], R).
R = true.
?- evaluare(imp(v3, si(v1, sau(v2, v3))), [v1], R).
R = false.
```

4.2 Sintaxa LP

Până acum am lucrat cu semantica: am evaluat formule, despre cum le dăm "înțeles" . Acum schimbăm perspectiva și ne uităm la latura *sintactică*: nu ne mai interesează dacă o formulă este adevărată într-o evaluare, ci dacă poate fi *demonstrată formal* din axiome și din anumite ipoteze. Practic, nu ne mai folosim de funcții de evaluare, ci de reguli.

În LP, lucrul acesta se face printr-un sistem deductiv de tip Hilbert. El are foarte puține reguli, dar este suficient de puternic încât să poată exprima toate demonstrațiile de care avem nevoie.

4.2.1 Sistemul de deducție

Sistemul Hilbert pentru LP este format dintr-o mulțime de scheme de axiome și o singură regulă de deducție. Axiomele reprezintă, practic, niște formule de bază, pe care nu trebuie să le demonstrăm. Regula de deducție este regula prin care demonstrăm noi formule, de la formule deja demonstrate.

Fie φ, ψ, χ formule. Axiomele lui LP sunt:

(A1) $\varphi \to (\psi \to \varphi)$

(A2) $(\varphi \to (\psi \to \chi)) \to ((\varphi \to \psi) \to (\varphi \to \chi))$

(A3) $(\neg\psi \to \neg\varphi) \to (\varphi \to \psi)$

Regula de deducție este *Modus Ponens*:

$$\frac{\varphi,\ \varphi \to \psi}{\psi}$$

Adică, dacă am demonstrat deja φ și am demonstrat și implicația $\varphi \to \psi$, atunci avem voie să inferăm ψ. (Adică îl considerăm pe ψ) demonstrat. Aceasta este singura regulă de deducție a sistemului.

4.2.2 Γ-teoreme

De obicei nu vrem să demonstrăm formule doar din axiome, ci și pornind de la anumite ipoteze suplimentare. Acesta este rolul mulțimii Γ: ea conține formulele pe care avem voie să le folosim ca puncte de plecare în demonstrație.

Definition 11. Fie Γ o mulțime de formule. Γ-teoremele sunt formulele definite astfel:

(T0) Orice axiomă este Γ-teoremă.

(T1) Orice formulă din Γ este Γ-teoremă.

(T2) Dacă φ și $\varphi \to \psi$ sunt Γ-teoreme, atunci și ψ este Γ-teoremă.

(T3) Numai formulele obținute prin aplicări finite ale regulilor (T0), (T1), (T2) sunt Γ-teoreme.

Dacă φ este Γ-teoremă, spunem și că φ este **dedusă** din ipotezele Γ.

Notații

Vom folosi notațiile standard:

$$\begin{aligned} Thm(\Gamma) &:= \text{mulțimea tuturor } \Gamma\text{-teoremelor}, \\ \Gamma \vdash \varphi &:\Leftrightarrow \varphi \in Thm(\Gamma), \\ \Gamma \vdash \Delta &:\Leftrightarrow \Gamma \vdash \delta \text{ pentru orice } \delta \in \Delta, \\ Thm &:= Thm(\emptyset), \\ \vdash \varphi &:\Leftrightarrow \emptyset \vdash \varphi. \end{aligned}$$

Definition 12. O formulă φ se numește **teoremă a lui LP** dacă $\vdash \varphi$.

Note (Inducție după Γ-teoreme)**.** Definiția inductivă a lui $Thm(\Gamma)$ ne dă și o metodă standard de demonstrație.

1. Dacă vrem să arătăm că orice Γ-teoremă are o proprietate P, este suficient să verificăm axiomele, formulele din Γ și închiderea lui P la Modus Ponens. (adică, orice formulă rezultată din MP) resprectă și ea proprietatea P.

2. Dacă vrem să arătăm că $Thm(\Gamma) \subseteq \Sigma$, este suficient să verificăm că toate axiomele sunt în Σ, toate formulele din Γ sunt în Σ, iar Σ este închisă la Modus Ponens.

Aceasta este exact analogia inducției pe care am folosit-o și la formule.

Proposition 10. Fie Γ, Δ mulțimi de formule. Atunci:

1. dacă $\Gamma \subseteq \Delta$, atunci $Thm(\Gamma) \subseteq Thm(\Delta)$;
2. $Thm \subseteq Thm(\Gamma)$;
3. dacă $\Gamma \vdash \Delta$, atunci $Thm(\Delta) \subseteq Thm(\Gamma)$;
4. $Thm(Thm(\Gamma)) = Thm(\Gamma)$.

Punctul (i) spune că mai multe ipoteze nu pot strica o demonstrație: dacă puteai demonstra ceva din Γ, poți demonstra și dintr-o mulțime mai mare. Punctul (ii) spune că orice teoremă rămâne demonstrabilă și atunci când adăugăm ipoteze. Punctele (iii) și (iv) formalizează ideea că o mulțime de formule poate fi înlocuită cu toate consecințele ei deductive fără să schimbăm ce mai poate fi demonstrat din ea.

4.2.3 Γ-demonstrații

Până acum am definit ce înseamnă să fie o formulă demonstrabilă din Γ, dar nu am spus încă explicit cum arată o demonstrație concretă. Pentru asta intro-

ducem noțiunea de secvență de formule.

Definition 13. O Γ-demonstrație este o secvență de formule

$$\theta_1, \theta_2, \ldots, \theta_n$$

astfel încât pentru orice $i \in \{1, \ldots, n\}$, una dintre următoarele condiții este satisfăcută:

1. θ_i este axiomă;
2. $\theta_i \in \Gamma$;
3. există $j, k < i$ astfel încât $\theta_k = \theta_j \to \theta_i$.

Pe scurt, o demonstrație este o secvență de formule pe care **le demonstrăm**. Condițiile 1 și 2 sunte clare, acestea ilustrând că într-o demonstrație putem scrie formule care sunt Γ-teoreme conform definiției. Mai complicată este condiția 3, care afirmă că o formulă este demonstrată a fi Γ-teoremă dacă există două formule care apar precedent în teoremă (θ_j și $\theta_j \to \theta_i$). Acest lucru reprezintă aplicarea Modus Ponens.

Lemma 1. Dacă $\theta_1, \ldots, \theta_n$ este o Γ-demonstrație, atunci $\Gamma \vdash \theta_i$ pentru orice $i \in \{1, \ldots, n\}$.

Proof. Demonstrăm prin inducție după i.

Pentru un i fix, dacă θ_i este axiomă, rezultă $\Gamma \vdash \theta_i$ din punctul (i) al propoziției anterioare. Dacă $\theta_i \in \Gamma$, rezultă $\Gamma \vdash \theta_i$ din punctul (ii). Dacă există $j, k < i$ cu $\theta_k = \theta_j \to \theta_i$, atunci prin ipoteza de inducție avem $\Gamma \vdash \theta_j$ și $\Gamma \vdash \theta_j \to \theta_i$, deci din punctul (iii) obținem $\Gamma \vdash \theta_i$. $\square$

Definition 14. Fie φ o formulă. O demonstrație din Γ a lui φ este o secvență ca mai sus a cărei ultimă formulă este φ.

Numărul n se numește **lungimea** demonstrației.

Proposition 11. Pentru orice mulțime de formule Γ și orice formulă φ, avem

$$\Gamma \vdash \varphi \quad \text{ddacă} \quad \text{există o } \Gamma\text{-demonstrație a lui } \varphi.$$

Practic, dacă o formulă este oΓ-teoremă înseamnă că există și o Γ-demonstrație pentru aceasta.

Exercițiu. Fie $\Gamma = \{\varphi, \varphi \to \psi\}$. Să se construiască o Γ-demonstrație a lui ψ.

Soluție. Luăm secvența:

$$\theta_1 = \varphi, \qquad \theta_2 = \varphi \to \psi, \qquad \theta_3 = \psi.$$

Primele două formule sunt în Γ, deci sunt permise prin regula (T1). Ultima formulă se obține din primele două prin Modus Ponens. Prin urmare, $\theta_1, \theta_2, \theta_3$ este o Γ-demonstrație a lui ψ, deci $\Gamma \vdash \psi$. $\square$

4.2.4 Proprietăți sintactice

Proposition 12. Pentru orice formulă φ, avem

$$\vdash \varphi \to \varphi.$$

Proof. Demonstrația standard în sistem Hilbert are cinci pași:

(1)	$\vdash (\varphi \to ((\varphi \to \varphi) \to \varphi))$ $\to ((\varphi \to (\varphi \to \varphi)) \to (\varphi \to \varphi))$	A2
(2)	$\vdash \varphi \to ((\varphi \to \varphi) \to \varphi)$	A1
(3)	$\vdash (\varphi \to (\varphi \to \varphi)) \to (\varphi \to \varphi)$	MP din (1), (2)
(4)	$\vdash \varphi \to (\varphi \to \varphi)$	A1
(5)	$\vdash \varphi \to \varphi$	MP din (3), (4)

Acesta este primul exemplu important de formulă care nu este axiomă, dar poate fi demonstrată din axiome prin aplicări succesive ale lui MP. $\square$

Theorem 4 (Teorema deducției)**.** Fie Γ o mulțime de formule și fie φ, ψ formule. Atunci

$$\Gamma \cup \{\varphi\} \vdash \psi \quad \text{ddacă} \quad \Gamma \vdash \varphi \to \psi.$$

Teorema deducției este unul dintre cele mai utile instrumente sintactice. Ea spune că, dacă putem demonstra ψ folosind în plus ipoteza φ, atunci putem „muta" acea ipoteză în stânga unei implicații și obținem o demonstrație pentru $\varphi \to \psi$ fără să mai presupunem φ.

Consecințe utile

Proposition 13. Pentru orice formule φ, ψ, χ, avem

$$\vdash (\varphi \to \psi) \to ((\psi \to \chi) \to (\varphi \to \chi)).$$

Proposition 14. Pentru orice mulțime de formule Γ și orice formule φ, ψ, χ, dacă

$$\Gamma \vdash \varphi \to \psi \qquad \text{și} \qquad \Gamma \vdash \psi \to \chi,$$

atunci

$$\Gamma \vdash \varphi \to \chi.$$

Proof. Știm din propoziția anterioară că

$$\vdash (\varphi \to \psi) \to ((\psi \to \chi) \to (\varphi \to \chi)).$$

Prin faptul că orice teoremă este și Γ-teoremă, avem

$$\Gamma \vdash (\varphi \to \psi) \to ((\psi \to \chi) \to (\varphi \to \chi)).$$

Aplicând MP cu cele două ipoteze din enunț, obținem succesiv

$$\Gamma \vdash (\psi \to \chi) \to (\varphi \to \chi)$$

și apoi

$$\Gamma \vdash \varphi \to \chi.$$

□

Note (Formule derivate utile)**.** În practică, după primele demonstrații nu mai reluăm de fiecare dată derivările de bază. Câteva rezultate utile, pe care merită să le rețineți, sunt:

1. $\vdash (\varphi \to (\psi \to \chi)) \to (\psi \to (\varphi \to \chi))$, adică putem permuta antecedentele.
2. $\{\psi, \neg\psi\} \vdash \varphi$, adică din contradicție putem demonstra orice.
3. $\vdash \neg\neg\varphi \to \varphi$ și $\vdash \varphi \to \neg\neg\varphi$, adică dubla negație funcționează în ambele sensuri.
4. $\vdash (\varphi \to \psi) \to (\neg\psi \to \neg\varphi)$, adică avem contrapozitia.
5. Dacă $\Gamma \cup \{\psi\} \vdash \varphi$ și $\Gamma \cup \{\neg\psi\} \vdash \varphi$, atunci $\Gamma \vdash \varphi$. Aceasta este forma sintactică a unei demonstrații prin cazuri.

Toate aceste formule se demonstrează în același stil: plecăm de la axiome, aplicăm MP și folosim frecvent Teorema deducției pentru a comprima demonstrațiile.

Proposition 15. Pentru orice formule φ, ψ, χ, avem:

1. $\{\varphi \wedge \psi\} \vdash \varphi$;
2. $\{\varphi \wedge \psi\} \vdash \psi$;
3. $\{\varphi, \psi\} \vdash \varphi \wedge \psi$;
4. $\{\varphi, \psi\} \vdash \chi$ ddacă $\{\varphi \wedge \psi\} \vdash \chi$;
5. $\vdash \varphi \wedge \psi \leftrightarrow \psi \wedge \varphi$.

Primele două puncte spun că dintr-o conjuncție putem extrage fiecare element al ei. Punctul (iv) spune că a lucra cu două ipoteze separate sau cu conjuncția lor este, deductiv, același lucru. Ultimul punct formalizează faptul

că ordinea elementelor din conjuncție nu contează.

Exercițiu. Să se demonstreze următoarele teoreme prin metoda deducției naturale:

(i) $(\varphi \to (\psi \to \chi)) \to (\psi \to (\varphi \to \chi))$;

(ii) $(\varphi \to \psi) \to (\varphi \to (\chi \to \psi))$.

Soluție. **Pentru formula (i):** $\vdash (\varphi \to (\psi \to \chi)) \to (\psi \to (\varphi \to \chi))$

Pentru a urmări demonstrația clar, prima coloană va reprezenta starea curentă (secvența deductivă), iar a doua coloană va indica justificarea (regula aplicată).

1)	$\{\varphi \to (\psi \to \chi), \psi, \varphi\} \vdash \varphi \to (\psi \to \chi)$	P1.53(ii)
2)	$\{\varphi \to (\psi \to \chi), \psi, \varphi\} \vdash \varphi$	P1.53(ii)
3)	$\{\varphi \to (\psi \to \chi), \psi, \varphi\} \vdash \psi \to \chi$	MP: 1), 2)
4)	$\{\varphi \to (\psi \to \chi), \psi, \varphi\} \vdash \psi$	P1.53(ii)
5)	$\{\varphi \to (\psi \to \chi), \psi, \varphi\} \vdash \chi$	MP: 3), 4)
6)	$\{\varphi \to (\psi \to \chi), \psi\} \vdash \varphi \to \chi$	Th. Ded. din 5)
7)	$\{\varphi \to (\psi \to \chi)\} \vdash \psi \to (\varphi \to \chi)$	Th. Ded. din 6)
8)	$\vdash (\varphi \to (\psi \to \chi)) \to (\psi \to (\varphi \to \chi))$	Th. Ded. din 7)

Pentru formula (ii): $\vdash (\varphi \to \psi) \to (\varphi \to (\chi \to \psi))$

1)	$\{\varphi \to \psi, \varphi\} \vdash \varphi \to \psi$	P1.53(ii)
2)	$\{\varphi \to \psi, \varphi\} \vdash \varphi$	P1.53(ii)
3)	$\{\varphi \to \psi, \varphi\} \vdash \psi$	MP: 1), 2)
4)	$\{\varphi \to \psi, \varphi\} \vdash \psi \to (\chi \to \psi)$	A1
5)	$\{\varphi \to \psi, \varphi\} \vdash \chi \to \psi$	MP: 3), 4)
6)	$\{\varphi \to \psi\} \vdash \varphi \to (\chi \to \psi)$	Th. Ded. din 5)
7)	$\vdash (\varphi \to \psi) \to (\varphi \to (\chi \to \psi))$	Th. Ded. din 6)

□